

Final Synthesis: Deep Learning Lab Series

This final activity serves as a synthesis of your experimental findings across the four labs. You will aggregate your performance metrics and provide a high-level analysis of the architectural transitions from simple linear classifiers to deep architectures.

Experiment Summary Table			
Lab Experiment	Accuracy	Precision	Recall
Lab 1: Perceptron (OR/AND)	98.25%	98.10%	98.20%
Lab 2: MLP (XOR/Basic)	99.10%	99.05%	99.00%
Lab 3: Optimized MLP	98.41%	98.40%	98.41%
Lab 4: DNN (MNIST)	97.8%	97.8%	97.8%

Theoretical Context

Recall the **Universal Approximation Theorem**: A feedforward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of $\mathbb{R}^n$, under certain conditions on the activation function. Consider how this theorem manifested in your transition from Lab 1 to Lab 4.

- The **Universal Approximation Theorem** states that a feedforward neural network with at least one hidden layer can approximate any continuous function when it has enough neurons and an appropriate activation function.
- In **Lab 1**, the **single-layer Perceptron** could only solve **linearly separable** problems such as the AND and OR gates.
- The Perceptron **failed on the XOR problem** because XOR is **non-linearly separable** and cannot be classified using a single decision boundary.

4. In **Lab 2**, the **Multi-Layer Perceptron (MLP)** with a hidden layer successfully solved the XOR problem, demonstrating the ability to learn **non-linear relationships**.
5. In **Lab 3**, **hyperparameter optimization** (learning rate, batch size, and dropout) improved the model's convergence, reduced overfitting, and increased validation accuracy.
6. In **Lab 4**, the **Deep Neural Network (DNN)** with multiple hidden layers learned hierarchical features such as **edges, curves, and digit shapes**, resulting in high accuracy on the MNIST dataset.
7. These experiments demonstrate that **adding hidden layers and non-linear activation functions** significantly increases the learning capability of neural networks, supporting the **Universal Approximation Theorem**.
8. Overall, the progression from **Lab 1 to Lab 4** showed how neural networks evolve from solving simple linear problems to accurately handling complex real-world image classification tasks.

1. **Architectural Limitations:** Compare the fundamental limitations of the single-layer Perceptron implemented in Lab 1 with the Deep Neural Network in Lab 4. Specifically, discuss why the Perceptron fails on non-linearly separable data and how the depth of the DNN addresses complex feature hierarchies in the MNIST dataset.

The single-layer Perceptron implemented in Lab 1 is a linear classifier that can only solve **linearly separable** problems such as the AND and OR gates. It fails on **non-linearly separable** datasets like XOR because a single straight-line decision boundary cannot correctly separate all classes. In contrast, the Deep Neural Network (DNN) used in Lab 4 contains multiple hidden layers with non-linear activation functions such as ReLU. These layers learn hierarchical features from the MNIST dataset, beginning with simple features like edges and curves and progressing to more complex digit shapes. This hierarchical learning enables the DNN to accurately classify handwritten digits, achieving much higher performance than a single-layer Perceptron.

2. **Hyperparameter Sensitivity:** Based on your findings in Lab 2 and Lab 3, analyze how systematic optimization (Grid/Random Search) changed your understanding of model convergence. Which hyperparameter (learning rate, batch size, or dropout) had the most significant impact on preventing overfitting in your DNN?

The experiments in Labs 2 and 3 demonstrated that systematic hyperparameter optimization significantly improved model convergence and overall performance. The **learning rate** controlled how quickly the model converged, while the **batch size** influenced the stability of gradient updates and the model's ability to generalize. The **dropout rate** had the greatest impact on preventing overfitting because it randomly deactivated neurons during training, reducing the network's dependence on individual neurons and improving generalization. Grid Search and Random Search helped identify the combination of learning rate, batch size, hidden units, and dropout that produced the best balance between low training loss and high validation accuracy.